FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	**Kelzo Marcel O. Lumabas**
Section:	**AN42**
Professor:	**Ms. Crisola G. Tan**
Date:	**October 5, 2026**

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.

5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:
   - Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
   - Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:
   - MS Word, Google Docs, or LibreOffice
   - Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:
   - Knowledge Representation & Reasoning
   - Case-Based Reasoning cycle
   - Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
   - Google Drive for shared documentation
   - GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases. Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✓ Similarity score or ranking

✓ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.
This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✓ "New case saved" or similar confirmation

✓ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description
- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with “theft involving electronic transfer.” System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:
The reasoning engine adjusts argument patterns to match the new technical context.
Learning:
After final verdict, the case is stored—enhancing the system’s knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System

A CBR system identifies the learning struggles of students.
Student submits a math solution with repeated errors.
System retrieves past students with similar error patterns (e.g., misusing formulas).
It adapts feedback and personalized lessons based on successful interventions.
Learning:
System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks ⇩

```
# ============================================================
# INTELLIGENT TUTORING SYSTEM
# CASE-BASED REASONING (CBR)
#
# Case Study 3: Intelligent Tutoring System
# ============================================================

# ============================================================
# PART 1 - CASE BASE
# ============================================================

# The case base contains previous student problems.
#
# Each case has:
# - topic
# - error
# - difficulty
# - solution

case_base = [

    {
        "problem": {
            "topic": "Fractions",
```

```
            "error": "Incorrect addition",
            "difficulty": "Easy"
        },
        "solution": "Review how to find common denominators and practice adding
fractions."
    },

    {
        "problem": {
            "topic": "Fractions",
            "error": "Incorrect subtraction",
            "difficulty": "Medium"
        },
        "solution": "Practice finding common denominators and subtracting
fractions."
    },

    {
        "problem": {
            "topic": "Algebra",
            "error": "Sign error",
            "difficulty": "Medium"
        },
        "solution": "Review positive and negative numbers and practice algebraic
expressions."
    },

    {
        "problem": {
            "topic": "Algebra",
            "error": "Wrong formula",
            "difficulty": "Hard"
        },
        "solution": "Review the appropriate algebraic formula and solve guided
examples."
    },

    {
        "problem": {
            "topic": "Geometry",
            "error": "Wrong formula",
            "difficulty": "Easy"
        },
```

```
        "solution": "Review basic area and perimeter formulas and complete simple
geometry exercises."
    }

]

# ============================================================
# PART 2 - DISPLAY THE CASE BASE
# ============================================================

def display_case_base():

    print("\n==========================================")
    print("CURRENT CASE BASE")
    print("==========================================")

    for i, case in enumerate(case_base, start=1):

        problem = case["problem"]

        print(f"\nCase {i}:")
        print(f"  Topic: {problem['topic']}")
        print(f"  Error: {problem['error']}")
        print(f"  Difficulty: {problem['difficulty']}")
        print(f"  Solution: {case['solution']}")

# ============================================================
# PART 3 - GET NEW STUDENT PROBLEM
# ============================================================

def get_new_problem():

    print("\n==========================================")
    print("NEW STUDENT PROBLEM")
    print("==========================================")

    print("\nAvailable topics:")
    print("1. Fractions")
    print("2. Algebra")
    print("3. Geometry")

    topic_choice = input("\nChoose a topic (1-3): ")
```

```
    if topic_choice == "1":
        topic = "Fractions"

    elif topic_choice == "2":
        topic = "Algebra"

    elif topic_choice == "3":
        topic = "Geometry"

    else:
        print("Invalid choice. Using Fractions.")
        topic = "Fractions"

    print("\nAvailable error types:")
    print("1. Incorrect addition")
    print("2. Incorrect subtraction")
    print("3. Sign error")
    print("4. Wrong formula")

    error_choice = input("\nChoose an error type (1-4): ")

    if error_choice == "1":
        error = "Incorrect addition"

    elif error_choice == "2":
        error = "Incorrect subtraction"

    elif error_choice == "3":
        error = "Sign error"

    elif error_choice == "4":
        error = "Wrong formula"

    else:
        print("Invalid choice. Using Incorrect addition.")
        error = "Incorrect addition"

    print("\nAvailable difficulty levels:")
    print("1. Easy")
    print("2. Medium")
    print("3. Hard")

    difficulty_choice = input("\nChoose difficulty (1-3): ")
```

```
    if difficulty_choice == "1":
        difficulty = "Easy"

    elif difficulty_choice == "2":
        difficulty = "Medium"

    elif difficulty_choice == "3":
        difficulty = "Hard"

    else:
        print("Invalid choice. Using Medium.")
        difficulty = "Medium"


    # Create the new problem dictionary
    new_problem = {
        "topic": topic,
        "error": error,
        "difficulty": difficulty
    }

    return new_problem

# ============================================================
# PART 4 - SIMILARITY ASSESSMENT
# ============================================================

def calculate_similarity(new_problem, old_problem):

    # Start with zero matching attributes
    score = 0

    # Check topic
    if new_problem["topic"] == old_problem["topic"]:
        score += 1

    # Check error type
    if new_problem["error"] == old_problem["error"]:
        score += 1

    # Check difficulty
    if new_problem["difficulty"] == old_problem["difficulty"]:
```

```
        score += 1

    # There are 3 attributes.
    # Convert the score into a percentage.
    similarity = (score / 3) * 100

    return similarity

# ============================================================
# PART 5 - RETRIEVE SIMILAR CASES
# ============================================================

def retrieve_cases(new_problem):

    results = []

    # Compare the new problem with every case
    for i, case in enumerate(case_base):

        similarity = calculate_similarity(
            new_problem,
            case["problem"]
        )

        results.append({
            "case_number": i + 1,
            "case": case,
            "similarity": similarity
        })

    # Sort from highest similarity to lowest similarity
    results.sort(
        key=lambda result: result["similarity"],
        reverse=True
    )

    return results

# ============================================================
# PART 6 - DISPLAY SIMILARITY RESULTS
# ============================================================

def display_similarity_results(results):
```

```
    print("\n========================================")
    print("SIMILARITY RESULTS")
    print("========================================")

    for result in results:

        print(
            f"Case {result['case_number']}: "
            f"{result['similarity']:.1f}% similarity"
        )

# ============================================================
# PART 7 - DISPLAY RETRIEVED CASE
# ============================================================

def display_retrieved_case(best_result):

    case = best_result["case"]
    problem = case["problem"]

    print("\n========================================")
    print("MOST RELEVANT PAST CASE")
    print("========================================")

    print(f"Similarity: {best_result['similarity']:.1f}%")

    print("\nPrevious Problem:")
    print(f"  Topic: {problem['topic']}")
    print(f"  Error: {problem['error']}")
    print(f"  Difficulty: {problem['difficulty']}")

    print("\nPrevious Solution:")
    print(f"  {case['solution']}")

# ============================================================
# PART 8 - ADAPT THE SOLUTION
# ============================================================

def adapt_solution(best_case, new_problem):

    # Get the previous solution
    original_solution = best_case["solution"]
```

```
    # Start with the previous solution
    adapted_solution = original_solution

    # Add personalized instructions based on difficulty
    if new_problem["difficulty"] == "Easy":

        adapted_solution += (
            " Start with simple guided examples "
            "before moving to harder exercises."
        )

    elif new_problem["difficulty"] == "Medium":

        adapted_solution += (
            " Complete 5 practice problems "
            "with moderate difficulty."
        )

    elif new_problem["difficulty"] == "Hard":

        adapted_solution += (
            " Complete challenging practice problems "
            "and review the solution step-by-step."
        )

    # Add feedback based on the type of error
    if new_problem["error"] == "Sign error":

        adapted_solution += (
            " Pay special attention to positive "
            "and negative signs."
        )

    elif new_problem["error"] == "Wrong formula":

        adapted_solution += (
            " Review how to identify the correct "
            "formula before solving."
        )

    elif new_problem["error"] == "Incorrect addition":
```

```
        adapted_solution += (
            " Check each addition step carefully."
        )

    elif new_problem["error"] == "Incorrect subtraction":

        adapted_solution += (
            " Check the subtraction steps and "
            "make sure the signs are correct."
        )

    return adapted_solution

# ============================================================
# PART 9 - DISPLAY ADAPTED SOLUTION
# ============================================================

def display_adapted_solution(solution):

    print("\n==========================================")
    print("ADAPTED FINAL SOLUTION")
    print("==========================================")

    print(solution)

# ============================================================
# PART 10 - RETAIN NEW CASE
# ============================================================

def retain_case(new_problem, final_solution):

    new_case = {
        "problem": new_problem.copy(),
        "solution": final_solution
    }

    # Add the new case to the case base
    case_base.append(new_case)

    print("\n==========================================")
    print("RETAIN STAGE")
    print("==========================================")
```

```
    print("✓ New case saved successfully!")
    print("✓ The case base has been updated.")

# ============================================================
# PART 11 - DISPLAY NEW CASE BASE
# ============================================================

def display_updated_case_base():

    print("\n==========================================")
    print("UPDATED CASE BASE")
    print("==========================================")

    for i, case in enumerate(case_base, start=1):

        problem = case["problem"]

        print(
            f"\nCase {i}: "
            f"{problem['topic']} - "
            f"{problem['error']} - "
            f"{problem['difficulty']}"
        )

        print(f"Solution: {case['solution']}")

# ============================================================
# MAIN PROGRAM
# ============================================================

print("==========================================")
print("INTELLIGENT TUTORING SYSTEM")
print("CASE-BASED REASONING")
print("==========================================")

print("\nThis system uses previous student cases")
print("to recommend a personalized learning solution.")

# Show original case base
display_case_base()

# ------------------------------------------------------------
# STEP 1: Get new problem
```

```
# ------------------------------------------------------------

new_problem = get_new_problem()

print("\n==========================================")
print("NEW PROBLEM ENTERED")
print("==========================================")

print(f"Topic: {new_problem['topic']}")
print(f"Error: {new_problem['error']}")
print(f"Difficulty: {new_problem['difficulty']}")

# ------------------------------------------------------------
# STEP 2: RETRIEVE
# ------------------------------------------------------------

results = retrieve_cases(new_problem)

display_similarity_results(results)

# ------------------------------------------------------------
# STEP 3: Get most similar case
# ------------------------------------------------------------

best_result = results[0]

display_retrieved_case(best_result)

# ------------------------------------------------------------
# STEP 4: REUSE + ADAPT
# ------------------------------------------------------------

final_solution = adapt_solution(
    best_result["case"],
    new_problem
)

display_adapted_solution(final_solution)

# ------------------------------------------------------------
# STEP 5: RETAIN
# ------------------------------------------------------------
```

```
retain_case(
    new_problem,
    final_solution
)

# ------------------------------------------------------------
# STEP 6: Display updated case base
# ------------------------------------------------------------

display_updated_case_base()

print("\n==========================================")
print("CBR PROCESS COMPLETE")
print("==========================================")

print("\nThe system has:")
print("1. Retrieved a similar past case.")
print("2. Reused its previous solution.")
print("3. Adapted the solution to the new problem.")
print("4. Retained the new case in the case base.")
```

Reflection:

I developed the Intelligent Tutoring System without a pair by designing the case base, implementing the similarity calculation, and creating the retrieval, adaptation, and retention functions. One challenge I encountered was determining a simple but understandable method for comparing a new student problem with previous cases. I solved this by assigning one point to each matching attribute, including the topic, error type, and difficulty. I also had to make sure that the retrieved solution was adapted instead of simply copied from the previous case. With limited time through this activity, I learned how Case-Based Reasoning uses previous experiences to solve new problems. I learned that the Retrieve, Reuse, Adapt, and Retain stages work together to make the system more useful over time. This activity also has helped me understand how CBR can be applied to personalized mathematics learning, which may be helpful for my current thesis that involves math learning too.